

GUÍA PRÁCTICA #4

NOTA: Revisar y comprender las Guías Prácticas #1, #2 y #3.

OPERACIONES: Construye una aplicación de autenticación de usuarios.

PASO #1: Instalación de Laravel

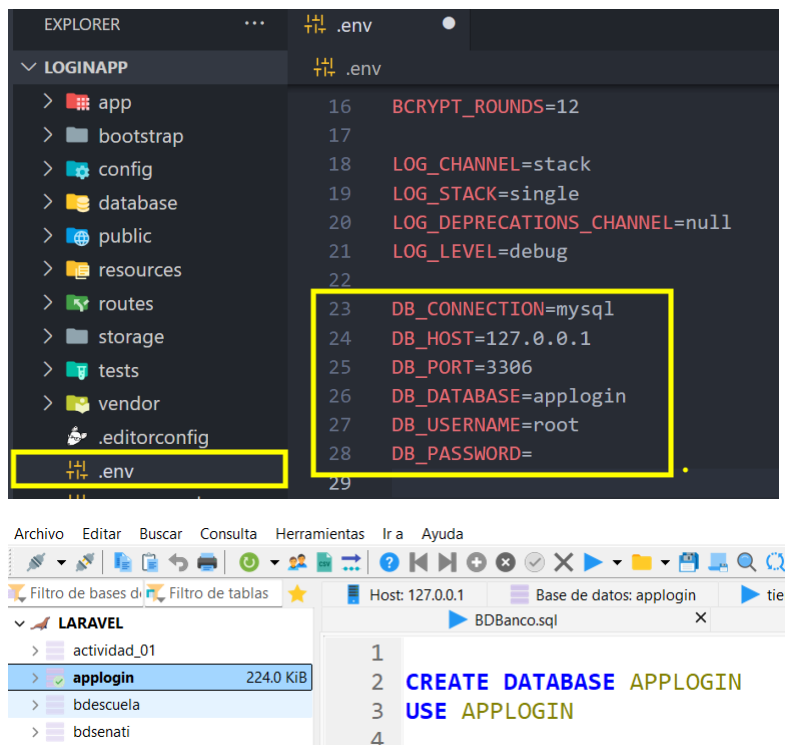
Iniciamos, creando un Proyecto en Laravel. Para ello, ejecutamos el siguiente código:

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs
$ composer create-project --prefer-dist laravel/laravel loginApp
```

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs
$ cd loginApp
```

PASO #2: Configuración de la base de datos.

Configurar la base de datos en el archivo **.env** de tu proyecto. Aquí debes colocar los detalles de tu conexión a la base de datos (**Crear la Base de Datos “applogin”**).



PASO #3: Realizar la migración.

Ejecuta las migraciones para generar las tablas necesarias, incluidas las de autenticación.

Ejecutar el siguiente código:

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/loginApp
$ php artisan migrate

INFO Preparing database.

Creating migration table ..... 31.73ms DONE

INFO Running migrations.

0001_01_01_000000_create_users_table ..... 77.38ms DONE
0001_01_01_000001_create_cache_table ..... 15.98ms DONE
0001_01_01_000002_create_jobs_table ..... 52.89ms DONE
```

PASO #4: Autenticación manual con sesiones.

Laravel ofrece una autenticación de usuarios por defecto que puedes usar. Para ello, ejecuta el siguiente comando:

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/loginApp
$ composer require laravel/ui
php artisan ui vue --auth
npm install && npm run dev

81 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

INFO No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
Using version ^4.6 for laravel/ui

The [Controller.php] file already exists. Do you want to replace it? (yes/no) [yes]
> yes
```

PASO #5: Corregir error. Forzar la instalación de dependencias.

Seguramente se mostrará un error relacionado con la versión de "vite".

```
npm ERR! code ERESOLVE
npm ERR! ERESOLVE unable to resolve dependency tree
npm ERR!
npm ERR! While resolving: undefined@undefined
npm ERR! Found: vite@7.1.5
npm ERR! node_modules/vite
npm ERR!   dev vite@"^7.0.4" from the root project
npm ERR!
npm ERR! Could not resolve dependency:
npm ERR! peer vite@"^4.0.0 || ^5.0.0" from @vitejs/plugin-vue@4.6.2
npm ERR! node_modules/@vitejs/plugin-vue
npm ERR!   dev @vitejs/plugin-vue@"^4.5.0" from the root project
npm ERR!
```



Si prefieres seguir con la versión más reciente de vite, puedes intentar forzar la resolución de dependencias mediante el comando `--legacy-peer-deps`, que ignorará los conflictos de dependencias peer:

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/loginApp
$ npm install --legacy-peer-deps
```

PASO #6: Autenticación básica HTTP para ciertas rutas API.

Para proteger algunas rutas de la API con autenticación básica HTTP, puedes utilizar el middleware `auth.basic` que Laravel ya proporciona. Por ejemplo:

En `routes/api.php`, define las rutas que requieren autenticación básica:

```
Route::middleware('auth.basic')->get('/user', function (Request $request)
{
    return $request->user();
});
```

Con esto, cualquier solicitud a esa ruta requerirá que el cliente pase las credenciales mediante autenticación básica.

NOTA: En este paso, seguramente no se ha creado el archivo `api.php` (en base a la experiencia previa, dar solución – Revisar Guía Práctica #1).

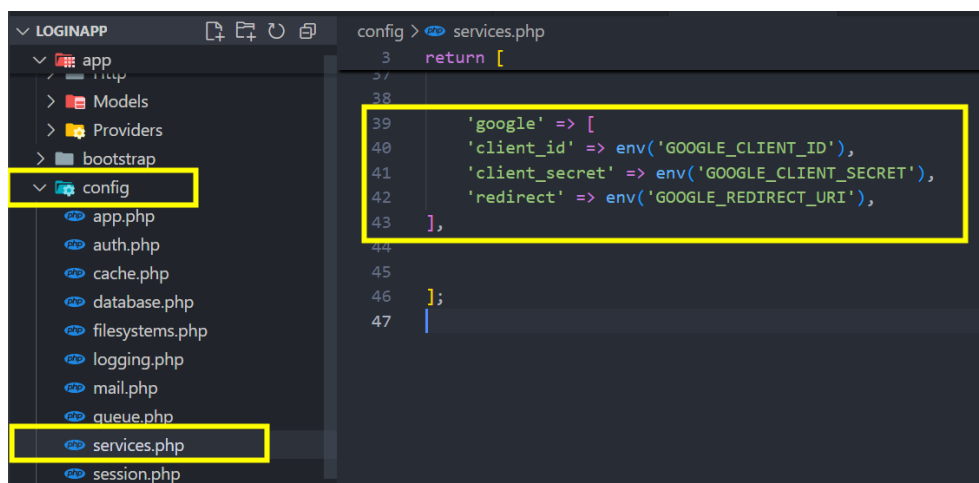
PASO #7: Autenticación social con Google.

Para implementar la autenticación con Google, usaremos **Laravel Socialite**, un paquete que facilita la integración con servicios de autenticación social.

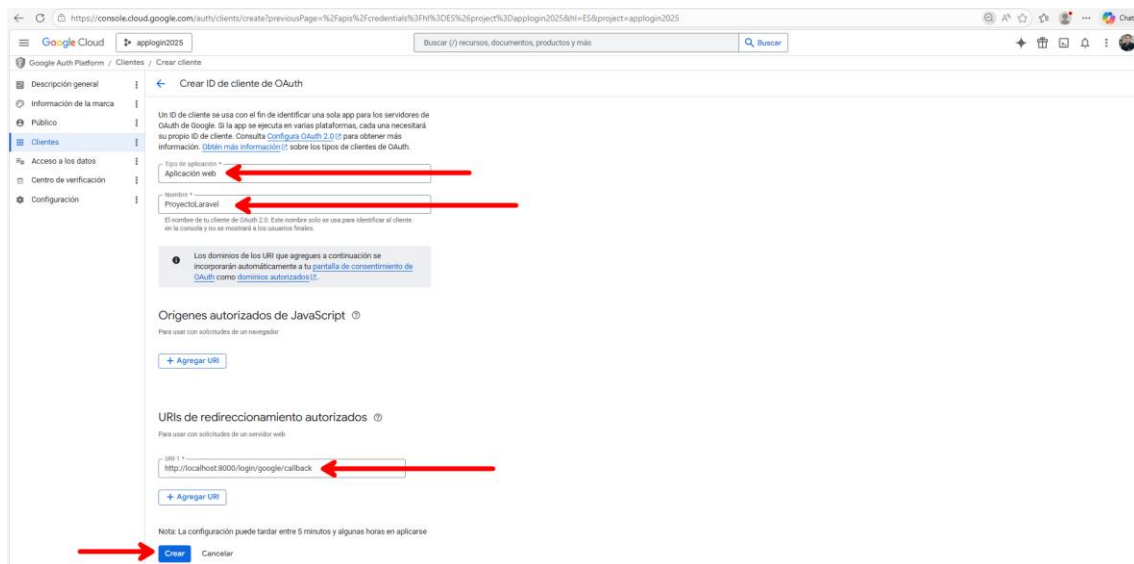
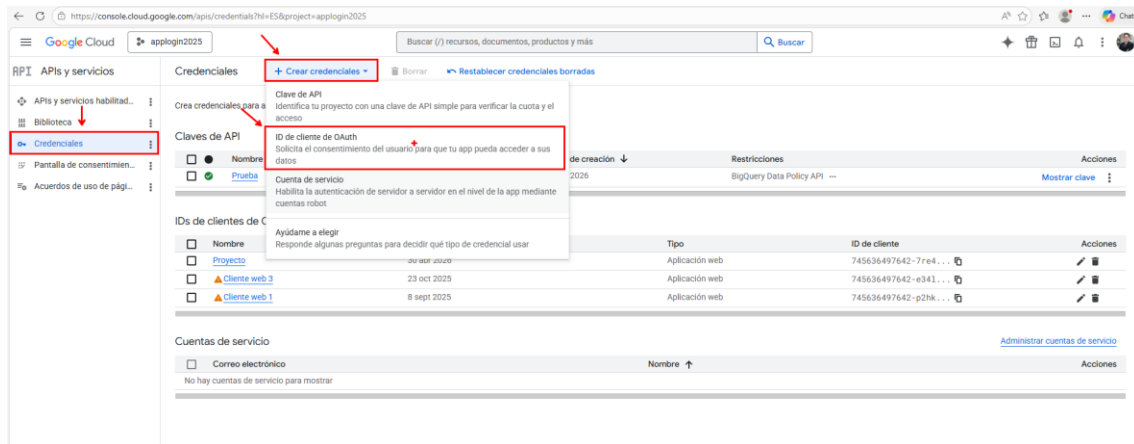
Instala Laravel Socialite:

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/loginApp
$ composer require laravel/socialite
```

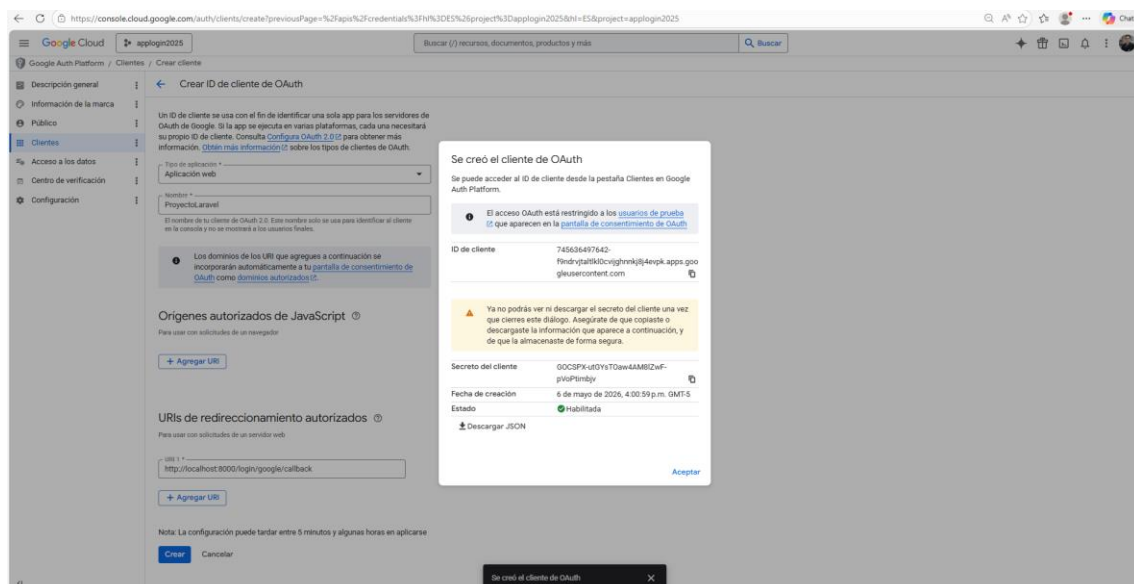
Configura Socialite en el archivo `config/services.php`:



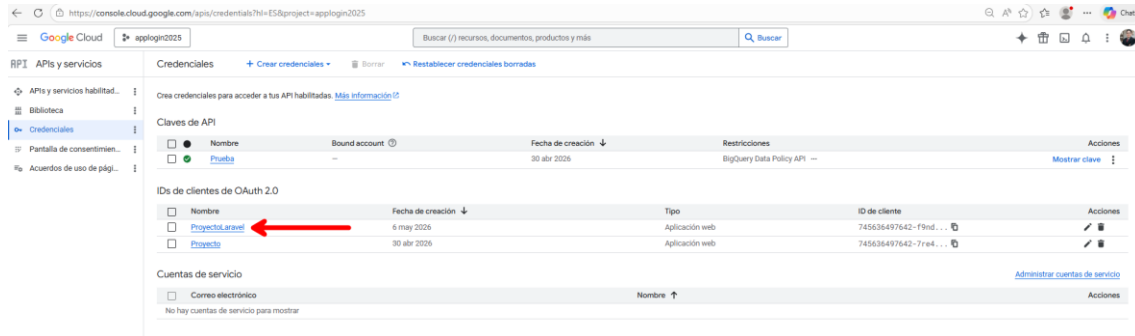
Obtén las credenciales de Google (ID de cliente y secreto) desde la [consola de desarrolladores de Google](#).



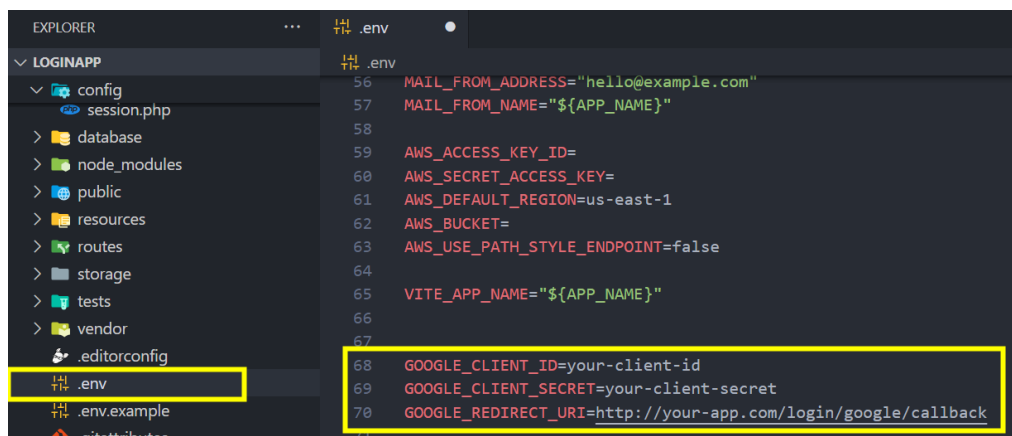
Se va a crear las credenciales de OAuth, guardarlo en un Bloc de notas.



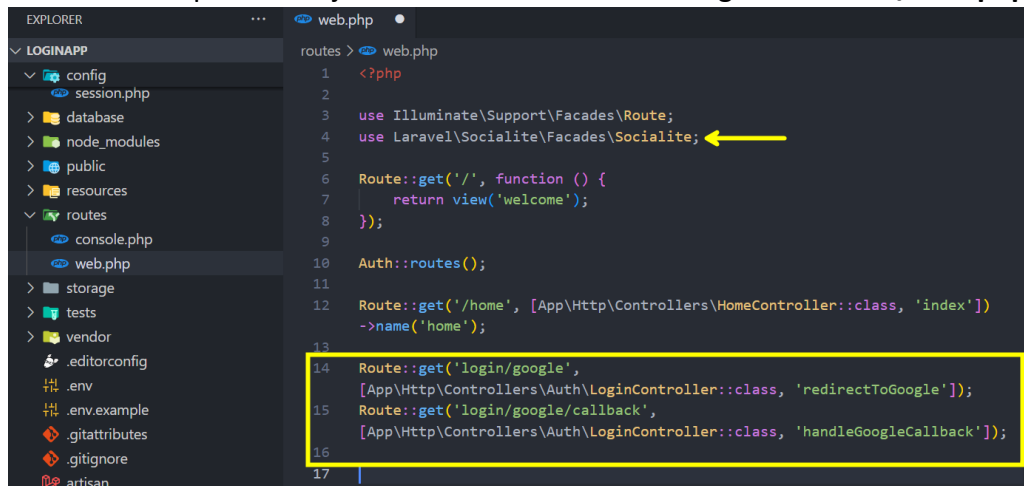
¡Listo! Ya creamos el cliente de OAuth de Google.



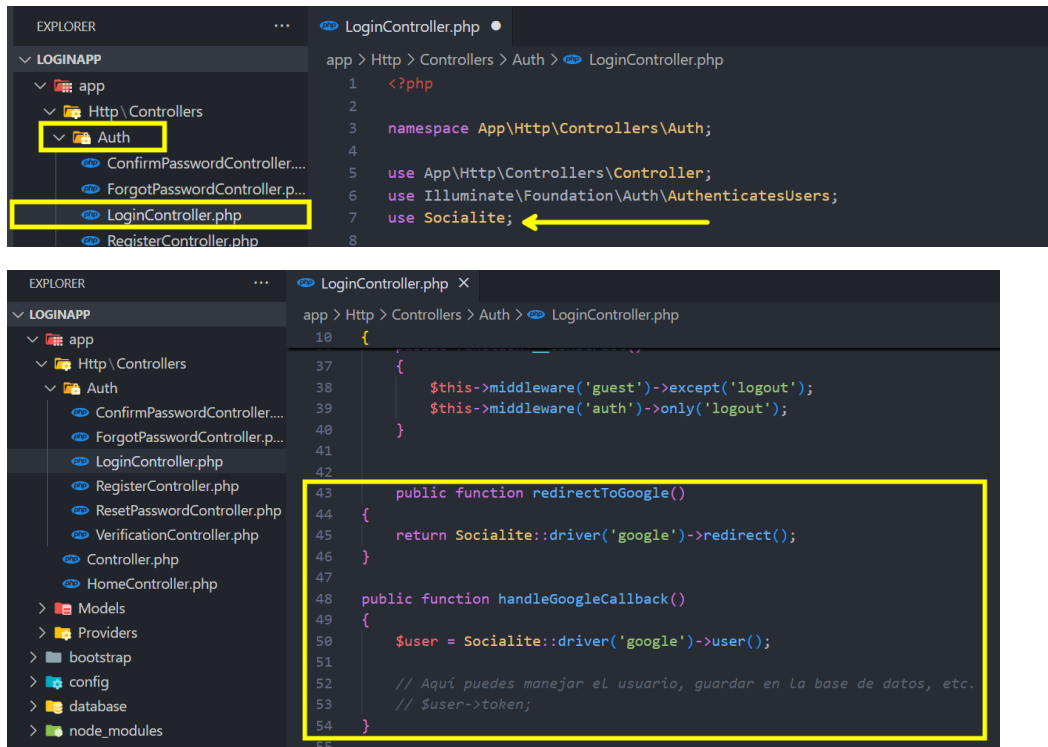
Agrega las credenciales de Google, el cual copiaste en el Bloc de notas, al archivo .env:



Crea una ruta para manejar el inicio de sesión con Google. En **routes/web.php**:



En el controlador LoginController, agrega los métodos para redirigir a Google y manejar la respuesta:



```

EXPLORER
  LOGINAPP
    app
      Http
        Controllers
          Auth
            ConfirmPasswordController.php
            ForgotPasswordController.php
            LoginController.php
            RegisterController.php
  ...

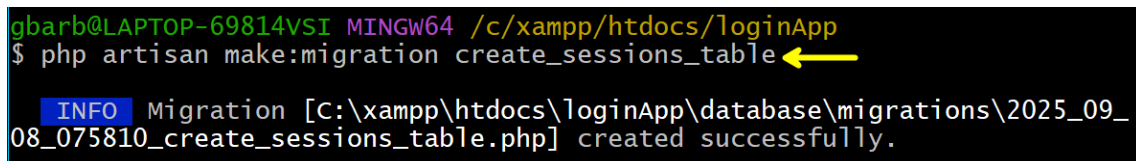
LoginController.php
  app > Http > Controllers > Auth > LoginController.php
  1  <?php
  2
  3  namespace App\Http\Controllers\Auth;
  4
  5  use App\Http\Controllers\Controller;
  6  use Illuminate\Foundation\Auth\AuthenticatesUsers;
  7  use Socialite;
  8

  10 {
  11
  12     // ...
  13
  14     public function redirectToGoogle()
  15     {
  16         return Socialite::driver('google')->redirect();
  17     }
  18
  19     public function handleGoogleCallback()
  20     {
  21         $user = Socialite::driver('google')->user();
  22
  23         // Aquí puedes manejar el usuario, guardar en la base de datos, etc.
  24         // $user->token;
  25     }
  26
  27     // ...
  28 }
  
```

PASO #7: Validación de sesión en múltiples dispositivos.

Laravel ya maneja la sesión de usuarios en múltiples dispositivos, pero puedes extender esto añadiendo una tabla para rastrear los dispositivos y las sesiones.

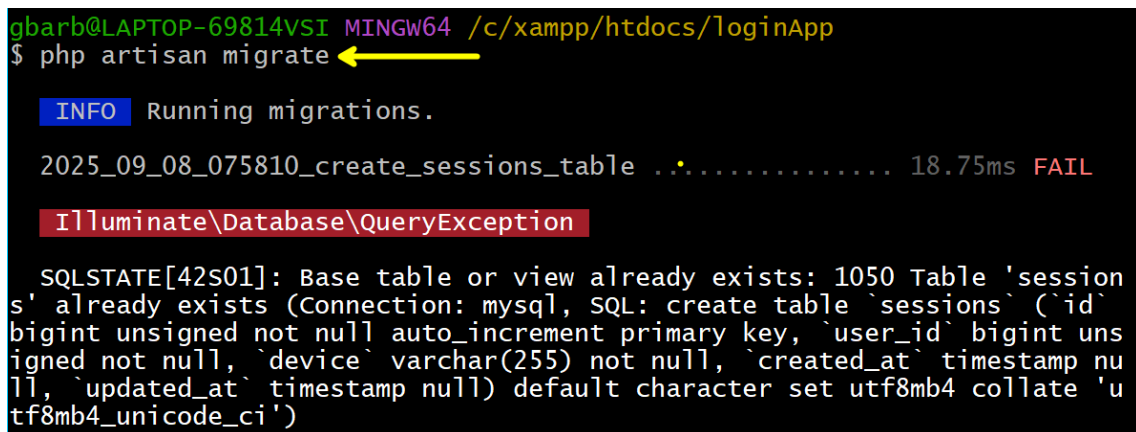
Crea una migración para la tabla sessions:



```

gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/loginApp
$ php artisan make:migration create_sessions_table
INFO Migration [C:\xampp\htdocs\loginApp\database\Migrations\2025_09_08_075810_create_sessions_table.php] created successfully.
  
```

Ejecutar la migración:



```

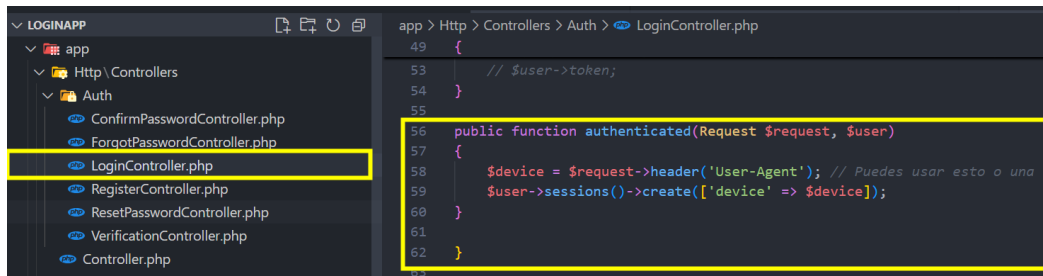
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/loginApp
$ php artisan migrate
INFO Running migrations.
2025_09_08_075810_create_sessions_table .. 18.75ms FAIL
Illuminate\Database\QueryException
SQLSTATE[42S01]: Base table or view already exists: 1050 Table 'sessions' already exists (Connection: mysql, SQL: create table `sessions` (`id` bigint unsigned not null auto_increment primary key, `user_id` bigint unsigned not null, `device` varchar(255) not null, `created_at` timestamp null, `updated_at` timestamp null) default character set utf8mb4 collate 'utf8mb4_unicode_ci')
  
```



NOTA: En este paso, seguramente saldrá el error

"Illuminate\Database\QueryException" (en base a la experiencia previa, dar solución).

En el controlador de autenticación, puedes registrar los dispositivos de los usuarios al iniciar sesión:



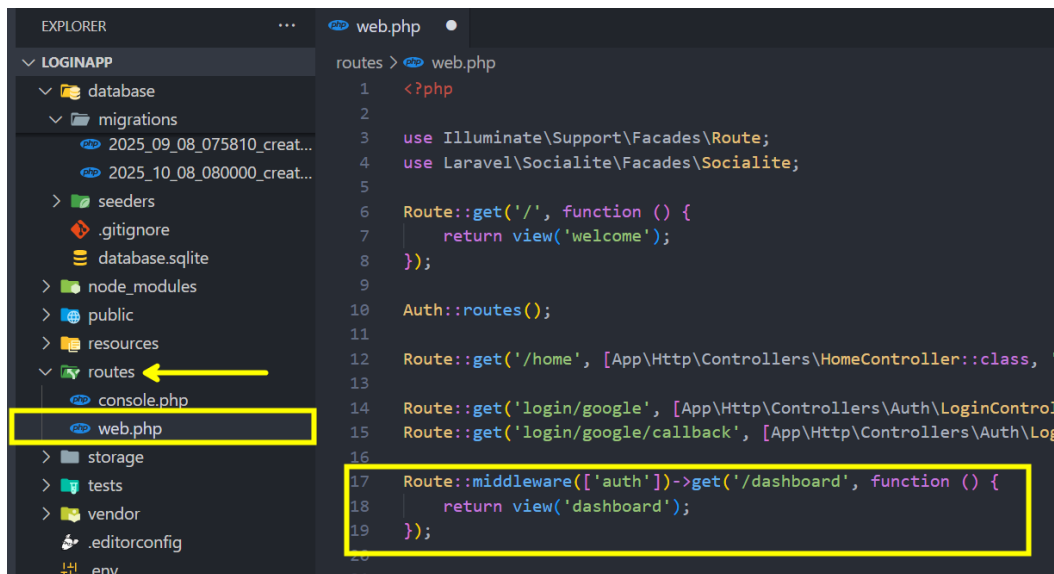
```

app > Http > Controllers > Auth > LoginController.php
49 {
50     // $user->token;
51 }
52
53 public function authenticated(Request $request, $user)
54 {
55     $device = $request->header('User-Agent'); // Puedes usar esto o una
56     $user->sessions()->create(['device' => $device]);
57 }
58
59 }
60
61 }
62

```

PASO #8: Protección contra accesos no autorizados.

Laravel ya incluye mecanismos de protección contra accesos no autorizados. Para asegurarte de que solo los usuarios autenticados puedan acceder a ciertas rutas, utiliza el middleware auth en tus rutas:



```

EXPLORER
LOGINAPP
  database
  migrations
  2025_09_08_075810_creat...
  2025_10_08_080000_creat...
  seeders
  .gitignore
  database.sqlite
  node_modules
  public
  resources
  routes
  console.php
  web.php
  storage
  tests
  vendor
  .editorconfig
  .env

routes > web.php
1 <?php
2
3 use Illuminate\Support\Facades\Route;
4 use Laravel\Socialite\Facades\Socialite;
5
6 Route::get('/', function () {
7     return view('welcome');
8 });
9
10 Auth::routes();
11
12 Route::get('/home', [App\Http\Controllers\HomeController::class,
13
14 Route::get('login/google', [App\Http\Controllers\Auth\LoginContro
15 Route::get('login/google/callback', [App\Http\Controllers\Auth\Log
16
17 Route::middleware(['auth'])->get('/dashboard', function () {
18     return view('dashboard');
19 });
20
21

```

PASO #9: Ejecutar el servidor.

```

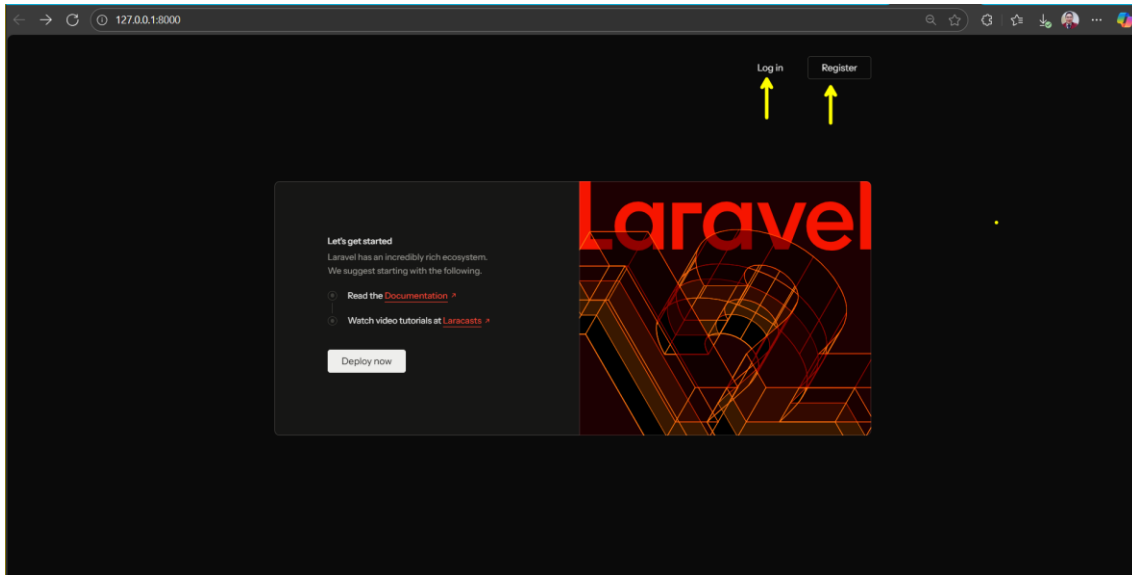
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/loginApp
$ php artisan serve

INFO Server running on [http://127.0.0.1:8000].

Press Ctrl+C to stop the server

```





PASO #10: Aplicar S/ 1.00 de CSS a la página de Login y Register.

